

# CS 521: Fall 2026

## Homework 1

Atharva Bhatt ([bhatt13@illinois.edu](mailto:bhatt13@illinois.edu))

### Problem 1. (FSGM attack)

1 (10 points). The file at <https://github.com/nalinwadhwa02/CCS521FA26HW/blob/main/hw1/fgsm.py> provides with an implementation of a simple neural network in PyTorch (<https://pytorch.org/get-started/locally/>). Follow the instructions there (you may have to read some PyTorch documentation) and complete the code to create a targeted FGSM attack. Write the relevant snippet in your submission and report on the results.

For the targeted FGSM attack, the perturbation is given by:

$$\eta = \epsilon \cdot \text{sign}(\nabla_x \text{loss}_t(x)) \quad \text{loss}_t(x) = -\log(f_\theta(x)_t)$$

where  $t$  is the target class. Subsequently, the perturbed input  $x'$  is,

$$x' = x - \eta$$

Implementing this in code, we get the following code snippet.

```
# your code here
# adv_x should be computed from x according to the fgsm-style perturbation such that the new class
# hint: you can compute the gradient of the loss w.r.t to x as x.grad
adv_x = x - eps*torch.sign(x.grad)
```

### Results:

The original class was:

Original Class: 2

After the FGSM attack:

```
new_class = N(adv_x).argmax(dim=1).item()
print("New Class: ", new_class)
assert(new_class == t)
# it is not enough that adv_x is classified as t. We also need to make sure it is 'close' to the o
print(torch.norm((x-adv_x), p=float('inf')).data)
assert( torch.norm((x-adv_x), p=float('inf')) <= epsReal)

... New Class: 0
tensor(0.5000)
```

The complete solution notebook can be found here: [\[Link\]](#)

2 (10 points). In the previous part you did a targeted attack on a network where the target class  $t$  (for the adversarial input) was 0. For this part, repeat the attack for the target class  $t = 1$ . If you were successful, report your results. If unsuccessful, explain what is happening: does this mean that the FGSM attack is ineffective for the given network? Play around with the parameters or use other techniques to find a close enough  $x'$  that fools the classifier. Report your results, including the method you used and the norm of the difference between your adversarial input  $x'$  and the given input  $x$ .

The FGSM attack (with  $\epsilon = 0.5$ ) used in the previous part was unable to find an *adversarial*  $x$  for the target class  $t = 1$ .

```

▶ epsReal = 0.5
  eps = epsReal - 1e-7
  adv_x = x - eps*torch.sign(x.grad)

new_class = N(adv_x).argmax(dim=1).item()
print("New Class: ", new_class)

New Class: 2

```

This means that a single-step FGSM attack with  $\epsilon = 0.5$  was unsuccessful in finding an *adversarial*  $x$  ( $adv\_x$ ). Thus, we resort to two approaches to find  $adv\_x$ .

### Approach 1: Epsilon Sweep

In this approach, we start with a small value of epsilon and increase it gradually until the neighborhood is sufficiently large to find  $adv\_x$  for the target  $t$  using a **single-step FGSM attack**. The  $eps$  was incremented in steps of  $eps\_del=0.001$ .

### Result:

```

▶ # Sweep epsReal with ea
  N_Sweep = int(1e4)
  eps_del = 0.001
  for i in range(N_Sweep):
    epsReal += eps_del
    eps = epsReal - 1e-7
    adv_x = x - eps*torch.sign(grad)
    new_class = N(adv_x).argmax(dim=1).item()
    if new_class == t:
      print("epsReal found:", epsReal)
      print("New Class: ", new_class)
      print("adv_x: ", adv_x)
      print("Loss: ", L(N(adv_x), torch.tensor([t], dtype=torch.long)))
      break

... epsReal found: 1.1379999999999855
  New Class: 1
  adv_x: tensor([[ -0.4796,  1.9555,  1.2642, -1.0839, -0.9112,  1.9785, -0.5987, -0.1582,
    1.9977, -0.5403]], grad_fn=<SubBackward0>)
  Loss: tensor(1.0963, grad_fn=<NLLLossBackward0>)

```

The smallest epsilon that enables the single-step FGSM attack to succeed is  $\epsilon = 1.138$ . The attack found  $adv\_x$  for the target class  $t = 1$ , and  $adv\_x$  is in the  $l_{inf}$  ball of  $x$  within  $\epsilon = 1.138$ .

## Approach 2: Targeted Iterative FGSM

In this approach, we aim to iteratively update  $adv\_x$  with  $\epsilon = 0.5$ , **as in the original approach**, using a step size  $\alpha = 0.25$ . We take steps until the model output equals the target class  $t$  or the  $max\_steps$  limit is reached. At each step, we compute the gradient of the previous  $adv\_x$  and iteratively step towards a better  $adv\_x$ . Also, we clip  $adv\_x$  within the  $l_{inf}$  ball of the original input  $x$ .

$$\eta = \alpha \cdot \text{sign}(\nabla_x \text{loss}_t(x)) \quad x'_{N+1} = \text{Clip}_{x,\epsilon}(x'_N - \eta)$$

## Result:

```

adv_x = x.clone().detach().requires_grad_()
model_output = N(adv_x).argmax(dim=1).item()
steps = 0
max_steps = int(1e2)

# Iterate until an adversarial x is found for target t.
while(model_output != t and steps < max_steps):
    loss = L(N(adv_x), torch.tensor([t], dtype=torch.long))
    loss.backward()
    grad = adv_x.grad
    adv_x = adv_x - alpha*torch.sign(grad) # FGSM step towards the target

    # Clip the adv_x to -eps to +eps neighbourhood of x.
    adv_x = x + torch.clamp(adv_x - x, -eps, eps)
    adv_x = adv_x.detach().requires_grad_()
    model_output = N(adv_x).argmax(dim=1).item()
    steps += 1

assert(model_output == t)
print("Adversarial X found in step #", steps)
print("adv_x: ", adv_x)
print("New Class: ", model_output)
print(torch.norm((x-adv_x), p=float('inf')).data)
assert(torch.norm((x-adv_x), p=float('inf')) <= epsReal)

... Adversarial X found in step # 18
adv_x: tensor([[0.1584, 0.8175, 0.6262, 0.3041, 0.2268, 0.8405, 0.0393, 0.4798, 1.3597,
0.0977]], requires_grad=True)
New Class: 1
tensor(0.5000)

```

$adv\_x$  is found in iteration #18. The new predicted class is the target class  $t = 1$ , and the found  $adv\_x$  is in the  $l_{inf}$  ball of  $x$  within  $\epsilon = 0.5$ .

The complete solution notebook can be found here: [\[Link\]](#)

## Problem 2. (Single and Multi-Norm Robustness with PGD attack)

1. ((15+15) point) In this problem, you will implement an untargeted  $l_\infty$  and  $l_2$  Projected Gradient Descent (PGD) attack [6] on a ResNet-18 model on the CIFAR10 dataset and record your observations. Also, you are provided with three model weights consisting of  $l_\infty$ ,  $l_2$ , and RAMP [4], which are trained to be empirically robust against  $l_\infty$ ,  $l_2$ , and multi-norm attacks. Report the standard accuracy and robust accuracy (the prediction accuracy by adversarial examples). Report any observations you may have on the accuracy differences among the provided models.

We implement the  $l_2$  and  $l_\infty$  PGD attacks and apply them to a PreActResNet18 on the CIFAR-10 dataset.

Now, one difference between iterative FGSM and PGD is that PGD uses a random starting point within the epsilon-ball of the original  $x$ . Thus, we implement it in the `random_start()` function.

```
# Function for a random starting point within the Epsilon ball.
def random_start(adv_x, eps, attack="pgd_linf"):
    if attack == "pgd_linf":
        adv_x = adv_x + torch.empty_like(adv_x).uniform_(
            -eps, eps
        )
        adv_x = torch.clamp(adv_x, min=0, max=1).detach()
    elif attack == "pgd_l2":
        delta = torch.empty_like(adv_x).normal_()
        d_flat = delta.view(adv_x.size(0), -1)
        n = d_flat.norm(p=2, dim=1).view(adv_x.size(0), 1, 1, 1)
        r = torch.zeros_like(n).uniform_(0, 1)
        delta *= r / n * eps
        adv_x = torch.clamp(adv_x + delta, min=0, max=1).detach()
    return adv_x
```

In this, we have two cases:

- L-inf PGD attack: In this, we can find a random starting point by taking a sample uniformly from  $[-\text{eps}, \text{eps}]$ , where  $\text{eps}$  is the epsilon.
- L-2 PGD attack: Here we first sample a delta from a normal distribution to get a direction of the movement and then we scale using the L-2 norm to restrict it to the epsilon-ball.

In both these cases, we finally clip the resulting  $\text{adv\_x}$  to the image domain.

The implementation of the L-2 and L-inf PGD attacks has been described below:

### 1. L-inf PGD attack

- Find a random start point in the L-inf epsilon-ball of input  $x$ .
- Repeat the following for steps  $k$ :

- i) Calculate the model output, the cross entropy loss, and the gradients for  $adv\_x$ .
- ii) Calculate the delta using  $delta = eps\_step * torch.sign(adv\_x.grad)$ .
- iii) Clip  $adv\_x + delta$  to the epsilon-ball of the original input  $x$ .
- iv) Clip the new  $adv\_x$  within the image domain.

```
def pgd_linf_untargeted(model, x, labels, k, eps, eps_step):
    model.eval()
    ce_loss = torch.nn.CrossEntropyLoss()
    adv_x = x.clone().detach()
    adv_x = random_start(adv_x, eps, attack="pgd_linf")
    for _ in range(k):
        adv_x.requires_grad_(True)
        model.zero_grad()
        output = model(adv_x)
        loss = ce_loss(output, labels)
        loss.backward()
        delta = eps_step * torch.sign(adv_x.grad)
        # find delta, clamp with eps
        adv_x = torch.clamp(adv_x + delta, x-eps, x+eps)
        # This clamp here is to clip to the image domain
        adv_x = torch.clamp(adv_x, min=0, max=1)
        adv_x = adv_x.detach()

    output = model(adv_x)
    return adv_x
```

## 2. L-inf PGD attack

- a) Find a random start point in the L-2 epsilon-ball of input  $x$ .
- b) Repeat the following for steps  $k$ :

```
for _ in range(k):
    adv_x.requires_grad_(True)
    output = model(adv_x)
    loss = ce_loss(output, labels)
    grad = torch.autograd.grad(
        loss, adv_x, retain_graph=False, create_graph=False
    )[0]
    grad_norms = (
        torch.norm(grad.view(batch_size, -1), p=2, dim=1)
        + eps_for_division
    )
    grad = grad / grad_norms.view(batch_size, 1, 1, 1)
    adv_x = adv_x.detach() + eps_step * grad

    delta = adv_x - x
    delta_norms = torch.norm(delta.view(batch_size, -1), p=2, dim=1)
    factor = eps / delta_norms
    factor = torch.min(factor, torch.ones_like(delta_norms))
    delta = delta * factor.view(-1, 1, 1, 1)

    # This clamp here is to clip to the image domain
    adv_x = torch.clamp(x + delta, min=0, max=1).detach()
```

- i) Calculate the model output, the cross entropy loss, and the gradients for  $adv\_x$ .
- ii) Scale down the gradients by their L-2 norm.
- iii) Calculate  $\delta = adv\_x - x$  and a projection scaling factor  $factor = eps / \delta\_norms$ , where  $eps$  is the epsilon and  $\delta\_norms$  are the L-2 norms of  $\delta$ .
- iv) Scale the delta using the projection scaling factor ( $factor$ ).
- v) Calculate the new  $adv\_x$  by clipping  $x + \delta$  to the image domain.  

$$adv\_x = torch.clamp(x + \delta, min=0, max=1).detach()$$

**Result:**

The following accuracies are calculated on three model weights,  $pretr\_Linf$ ,  $pretr\_L2$ , and  $pretr\_RAMP$ .

1. *Standard Accuracy*: The baseline accuracy with no attacks.
2. *Single Norm Robust Accuracy*:
  - a)  $pgd\_linf$ : The robust accuracy against L-inf untargeted PGD attack. The parameters of the attack are:  $eps = 8/255$ , number of steps  $k = 10$ ,  $eps\_step = eps/4$ .
  - b)  $pgd\_l2$ : The robust accuracy against L-2 untargeted PGD attack. The parameters of the attack are:  $eps = 0.75$ , number of steps  $k = 10$ ,  $eps\_step = eps/4$ .

The accuracy metrics are reported in the table below:

| Model Weights | Standard Accuracy | L-inf PGD Accuracy | L-2 PGD Accuracy |
|---------------|-------------------|--------------------|------------------|
| $pretr\_Linf$ | 82.80%            | <b>51.52%</b>      | 49.35%           |
| $pretr\_L2$   | <b>88.75%</b>     | 31.41%             | 54.89%           |
| $pretr\_RAMP$ | 81.19%            | 50.04%             | <b>59.87%</b>    |

**Observations:**

1. The models specifically trained on an attack perform best against it; i.e.,  $pretr\_Linf$  achieves the highest robust accuracy against the  $pgd\_linf$  attack, and  $pretr\_L2$  achieves the highest robust accuracy against the  $pgd\_l2$  attack.
2.  $pretr\_RAMP$  achieves standard, robust accuracies comparable to those of models trained specifically on those attacks.

2. (15 points) In this problem, you will implement the multi-norm robustness analysis [4] of those models. We focus on the union threat model  $\Delta = B_2(x, \epsilon_2) \cup B_\infty(x, \epsilon_\infty)$  which requires the model to be robust within the  $l_2, l_\infty$  adversarial regions simultaneously [1]. Union accuracy is then defined as the robustness against  $\Delta(i)$  for each  $x_i$  sampled from the dataset  $D$ . Report the robust accuracy under the multi-adversarial attacks on three models. Report any observations you may have on the accuracy differences among the provided models.

The multi-norm robustness is calculated by evaluating cases where the output for the  $adv\_x$  of both the  $l_2$  and  $l_\infty$  PGD attacks are correct (robust), i.e., the union accuracy, where both attacks are correct.

```
def test_model_on_multi_attacks(model, eps_linf=8./255., eps_l2=0.75):
    model.eval()
    tot_test, tot_acc = 0.0, 0.0
    for batch_idx, (x_batch, y_batch) in tqdm(enumerate(test_loader), total=len(test_loader), desc="Evaluating"):
        x_batch, y_batch = x_batch.to(device), y_batch.to(device)
        x_adv_linf = pgd_linf_untargeted(model, x_batch, y_batch, k=10, eps=eps_linf, eps_step=eps_linf/4)
        x_adv_l2 = pgd_l2_untargeted(model, x_batch, y_batch, k=10, eps=eps_l2, eps_step=eps_l2/4)

        ## calculate union accuracy: correct only if both attacks are correct

        out = model(x_adv_linf)
        pred_linf = torch.max(out, dim=1)[1]
        out = model(x_adv_l2)
        pred_l2 = torch.max(out, dim=1)[1]

        out = model(x_adv_linf)
        pred_linf = torch.max(out, dim=1)[1]

        out = model(x_adv_l2)
        pred_l2 = torch.max(out, dim=1)[1]

        correct = (pred_linf == y_batch) & (pred_l2 == y_batch)

        tot_acc += correct.sum().item()
        tot_test += y_batch.size(0)

    print('Robust accuracy %.5lf' % (tot_acc/tot_test), f'on multi attacks')
```

### Result:

| Model Weights     | Multi-Attack Accuracy |
|-------------------|-----------------------|
| <i>pretr_Linf</i> | 47.47%                |
| <i>pretr_L2</i>   | 31.48%                |
| <i>pretr_RAMP</i> | <b>50.01%</b>         |

### Observation:

1. *pretr\_RAMP*, i.e., the RAMP-trained model, performs the best against multi-attacks.
2. *pretr\_L2*, i.e., L-2 adversarial training, is the least effective in multi-attack robustness.

The complete solution notebook can be found here: [\[Link\]](#)

**Problem 3. (Adversarial robustness against unseen adversaries.)**

We have studied adversarial attacks in depth during lectures. But you may have wondered: why do these attacks persist, and how broad is their threat landscape? There is a stream of research examining robustness not just against known perturbations, but also against unforeseen adversarial distortions. For this problem, we ask you to read the paper “Testing Robustness Against Unforeseen Adversaries” [5]. You should write a critical, conference-style review of the paper, including the following parts:

1. Summary: Summarize the paper in your own words.

The paper argues that current adversarial robustness research primarily focuses on  $L_p$ -ball perturbations and that defenses are trained and evaluated against the same adversary. However, this does not reflect a real-world scenario, and thus, the paper introduces a framework for evaluating models against *unforeseen adversaries*, with benchmarks on ImageNet (ImageNet-UA) and CIFAR-10 (CIFAR-10-UA). The paper presents 19 intensity-tunable non- $L_p$  attacks (18 novel) and introduces a new metric, UA2 (Unforeseen Adversarial Accuracy), based on 8 of these *unforeseen attacks*. The proposed attacks have explicitly bounded perturbation sets and are differentiable, and thus Projected Gradient Descent (PGD) can be used iteratively to find an approximate solution to the worst-case scenario for a particular model and attack type, maximizing the model's loss. A human study confirms that the attacked images preserve their semantic content.

Additionally, they show that existing  $L_p$ -based training defenses don't generalize well to unforeseen attacks, with  $L_2$  training generalizing better than  $L_\infty$  training. Furthermore, they evaluate defense techniques beyond  $L_p$ -based adversarial training to improve robustness to unforeseen attacks, including data augmentation, Perceptual Adversarial Training (PAT), Variational Regularization (VR), a combination of VR and PAT, and more, and find that several of these methods outperform  $L_p$ -based training. They also find that general advances in computer vision, such as large-scale pretraining and model scale, improve robustness to unforeseen attacks. DINOv2 achieves the highest UA2 in their evaluation.

2. Strengths (at least 3): Elaborate on the strengths you see in the paper

1. **Unsaturated Benchmark.** The best model (DINOv2) achieves 27.7% UA2, while a human study achieves above 91.2% on the same set of attacked images. The benchmark is far from saturated and can track progress for a long time.
2. **Precisely defined, practical attacks.** Each attack has an explicit, hard-constrained perturbation set that is enforced by projection.

$$S_x^{A,\varepsilon} = \{A(x, \delta) \mid \|\delta\|_p \leq \varepsilon\}$$

Many prior non-LP attacks instead depend on solving a relaxed (penalty-based) optimization problem or on an auxiliary generative model. Here,  $\varepsilon$  works as a monotonic

strength controller. The attacks are also differentiable, independent of the dataset, and efficient (with overhead comparable to PGD), making the benchmark practical to adopt.

3. **Extensive validation of the attacks.** The evaluation builds on the guidelines outlined in prior literature (Carlini et al., 2019 [1]): white-box access, no gradient masking, and tuned hyperparameters for each attack. The gradient-based approach for optimizing attacks outperforms a random grid search by a large margin (7.2% vs. 74.1% model accuracy; lower is better for attacks).

Reference:

[1] Nicholas Carlini, Anish Athalye, Nicolas Papernot, Wieland Brendel, Jonas Rauber, Dimitris Tsipras, Ian J. Goodfellow, Aleksander Madry, and Alexey Kurakin. On evaluating adversarial robustness. CoRR, abs/1902.06705, 2019. URL <http://arxiv.org/abs/1902.06705>.

3. Weaknesses (at least 3): Mention the weaknesses of this paper and suggest possible fixes for the same.

1. **The UA2 metric hides per-attack failures.** UA2 averages accuracy across 8 attacks, and the evaluations (Table 2 of the original paper) show that accuracy varies widely across attacks, allowing an adversary to choose the worst attack for a model. An example of this shortcoming is seen in a claim they make, that L2 training beats Linf for unforeseen attacks (13.9% vs 10%), which comes entirely due to the difference in JPEG attack (39.7% vs 6.9%), and if we compute the average without JPEG, L2 and Linf are practically tied (10.2% vs 10.4%). Thus, UA2 alone cannot provide comprehensive details about the model behaviors.

**Solution:** More detailed per-attack breakdown and analysis.

2. **Insufficient justification for diversity in core attacks.** The paper claims that high UA2 shows generalization to "a diverse set of adversaries not seen at train time," but this is only tested on the 8 core attacks that comprise UA2. The core attack set is said to be selected "for diversity and effectiveness across model scale", but no diversity measure, selection procedure, or comparison with the remaining attacks is reported. Furthermore, the results of the remaining 11 attacks are not reported in the original paper, so it is never tested whether UA2 predicts robustness to other unseen attacks.

**Solution:** Evaluate the correlation between UA2 and accuracy across the remaining attacks, and conduct an ablation study to understand each attack's effect on UA2.

3. **Inconclusive defense findings.** The conclusions about which techniques improve unforeseen robustness are inconclusive, as the comparisons don't separate out individual factors. The paper compares models that differ in architecture, size, pretraining data, and training objective simultaneously, making it difficult to attribute any gains in UA2 to a single technique.

**Solution:** Run ablations that change one factor at a time (architecture, scale, pretraining, training method).

4. Extensions (at least 1): Although such a section is not included in conference reviews, mention possible extensions of this work in the context of contemporary research. E.g., Could this be extended to explain diverse kinds of adversarial attacks on LLMs? If so, how?

### **Learned Attacks to Extend the Unforeseen Robustness Benchmark**

The benchmark's attacks are expert-curated, so over time, the defenses will catch up and overfit to them. Consequently, we would need to design new attacks again to be “unforeseen”. Instead of designing them manually, their structure could be learned. For example, the Snow attack uses a fixed kernel and optimizes only the flake intensities (Gabor attack is similar). We could make the kernel learnable across the dataset, which would approximate the worst attack of that kind for that model while still keeping an explicit perturbation set and remaining differentiable. The objective would be to maximize the expected worst-case loss of a model that is already robust to the existing attacks. Constraints on the kernel (such as on kernel norm) and human validation can prevent changes that destroy the image. Once learned, the kernel is frozen, **producing a new attack family**. This would allow the benchmark to grow over time.

P.T.O.

**Bonus Research Question****Problem 5. (Option 2)**

In Problem 3, we saw that  $\ell_p$  defenses often fail to generalize to other unseen adversaries. In this bonus question, your task is to explore the relationship between  $\ell_p$  adversaries and other types of adversaries (<https://github.com/ddkang/advex-uar>). Recent works have also investigated improving multi-norm robustness [9, 4]. Specifically, can multi-norm training improve the UAR metric introduced by Kang et al. [5]? You may leverage implementations such as <https://github.com/uiuc-focal-lab/RAMP> to train a multi-norm robust model and then evaluate it on the benchmark of Kang et al. [5]. What insights can you draw from your results? For example, do you find evidence that carefully designed joint  $\ell_p$  adversarial training can improve UAR?

For this experiment, we reuse the PreActResNet18 with the weights "pretr\_Linf", "pretr\_L2", "pretr\_RAMP" (corresponding to Linf, L2, and multi-attack adversarial training) from **Problem 2**. We use the code from the paper "*Testing Robustness Against Unforeseen Adversaries*" (Kang et al. [1]), available at <https://github.com/centerforaisafety/adversarial-corruptions>, and evaluate the three models against the CIFAR-10-UA dataset to measure their robustness against unforeseen attacks.

Experiment parameters:

1. *BATCH\_SIZE* = 1000
2. *NUM\_BATCHES* = 1 (thus, we test on a total of  $1 * 1000 = 1000$  images)
3. *LEVEL* = "medium" (the intensity level of the attacks)

These parameters can be adjusted to balance evaluation accuracy and evaluation time.

**Results:**

The results from Problem 2 are shown below:

| Model Weights     | L-inf PGD Accuracy | L-2 PGD Accuracy | Multi-Attack Accuracy |
|-------------------|--------------------|------------------|-----------------------|
| <i>pretr_Linf</i> | <b>51.52%</b>      | 49.35%           | 47.47%                |
| <i>pretr_L2</i>   | 31.41%             | 54.89%           | 31.48%                |
| <i>pretr_RAMP</i> | 50.04%             | <b>59.87%</b>    | <b>50.01%</b>         |

P.T.O.

The performance of the models on unforeseen attacks is shown below:

| Model Weights     | gabor             | snow              | pixel             | jpeg              | kaleido<br>scope  | glitch            | wood              | elastic           | UA2               |
|-------------------|-------------------|-------------------|-------------------|-------------------|-------------------|-------------------|-------------------|-------------------|-------------------|
| <i>pretr_Linf</i> | 53.70<br>%        | <b>33.40</b><br>% | 15.50<br>%        | 48.70<br>%        | 33.90<br>%        | 62.60<br>%        | 33.10<br>%        | 47.00<br>%        | 40.99<br>%        |
| <i>pretr_L2</i>   | 55.60<br>%        | 18.60<br>%        | 14.50<br>%        | 59.80<br>%        | 23.70<br>%        | 62.20<br>%        | 37.90<br>%        | 39.00<br>%        | 38.91<br>%        |
| <i>pretr_RAMP</i> | <b>56.90</b><br>% | 32.40<br>%        | <b>24.50</b><br>% | <b>60.80</b><br>% | <b>34.50</b><br>% | <b>64.40</b><br>% | <b>50.40</b><br>% | <b>48.20</b><br>% | <b>46.51</b><br>% |

The model with the best performance in each category has been **highlighted**.

**Observations:** The model trained using *RAMP* with joint *Lp* adversarial training performs the **best across all the attacks** except one (*snow*). Between *pretr\_Linf* and *pretr\_L2*, the better performer varies across attacks.

**Conclusion:** The model trained using joint *Lp* adversarial training (*pretr\_RAMP*) has the best robustness against unforeseen attacks, as shown in the table above. This provides conclusive evidence for the hypothesis that carefully designed joint *Lp*-adversarial training can improve UA2.

The complete solution notebook can be found here: [\[Link\]](#)

---